

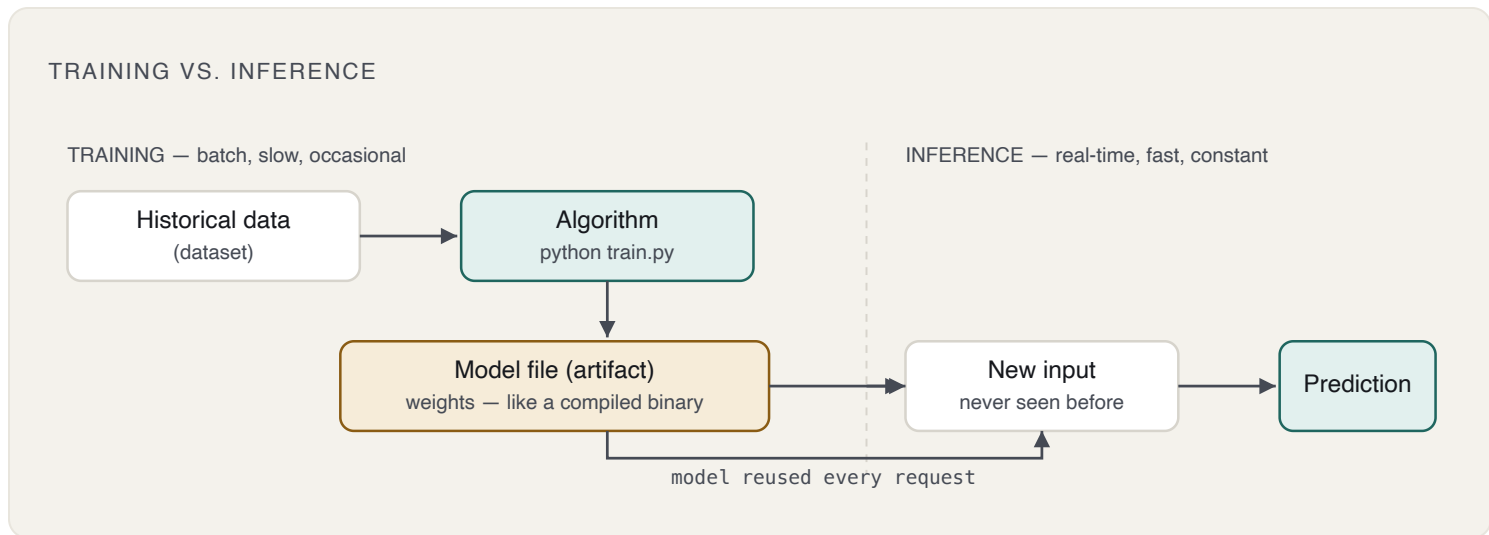
Running ML systems the way you already run everything else

Nine topics, condensed to the diagrams and tables you actually reach for mid-incident or mid-interview — from what a model even is, through drift, rollouts, GPU scheduling, and LLMOps.

Core idea: a normal app ships *code + config*. An ML app ships *code + data + config* → *trained model*. That one extra artifact — grown from data, not hand-written — is the source of nearly every MLOps concept below: versioning, feature stores, drift monitoring, evaluation gates, continuous training.

01 ML Basics for DevOps

A model is just another build artifact — its "source code" is data, run through an algorithm instead of a compiler.

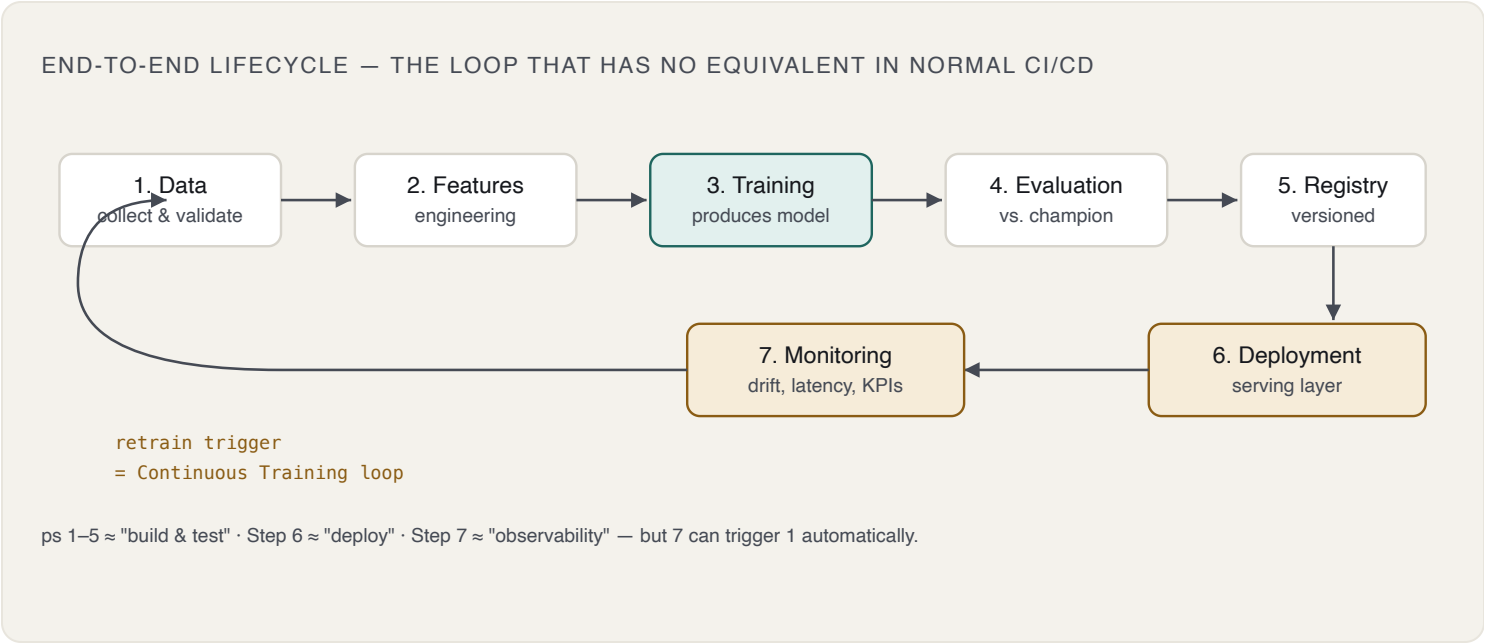


TERM	PLAIN MEANING	DEVOPS ANALOGY
Dataset	Table of past examples	Log/DB table you'd query for analytics
Feature	An input column (e.g. customer_age)	Config field / function parameter
Label	The correct answer for a past example	Expected value in a unit test assertion
Parameters / weights	Numbers learned during training	Compiled bytecode — functional, not human-read
Hyperparameters	Settings chosen before training starts	Build flags (-O2, --target=prod)
Ground truth	The real outcome, known later	Real incident matched against your alert

When to skip ML entirely: if you can write a clean, explicit if/else rule for it, do that instead — cheaper to build, debug, and operate. Reach for ML when the rule is too subtle to hand-write, drifts over time, or has no clean closed form.

02 Lifecycle & Artifacts

Software has one artifact to version (code). ML has three — and losing track of any one of them breaks reproducibility.



Three artifacts to version

- **Code**

Training script, feature logic, serving code — Git, as always.

- **Data**

The exact dataset trained on — DVC pointer files, or Delta Lake / Iceberg snapshots.

- **Model**

Weights + preprocessing + metadata — tracked in a model registry (MLflow, etc.).

MLOps maturity levels

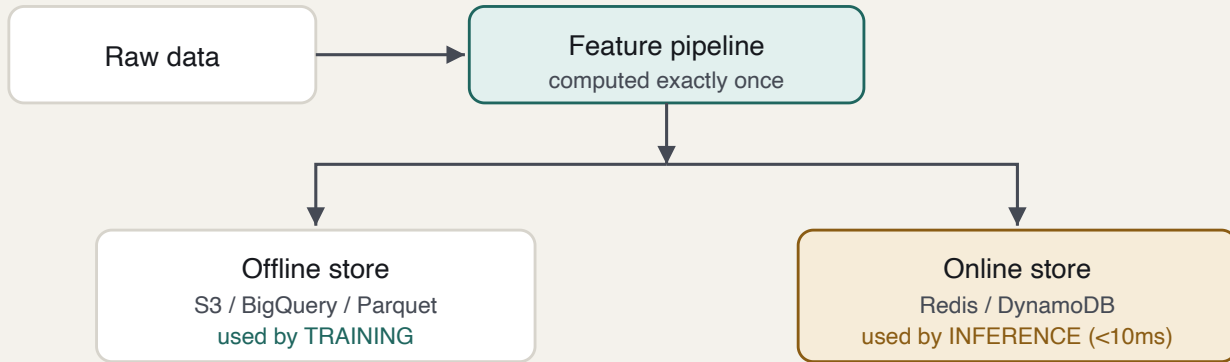
LEVEL	AUTOMATION	WHO TRIGGERS RETRAINING?
0 — Manual	None. Laptop → email → manual deploy.	A human, informally, rarely
1 — Pipeline automation	Data prep → train → eval is one automated pipeline	A human decides when + what to promote
2 — Full CI/CT/CD	Detects need, retrains, gates, promotes — end to end	The system, automatically

Champion = model currently live in production. **Challenger** = new candidate being evaluated to replace it — conceptually a canary deploy, but the metric is accuracy/precision/recall instead of error rate.

03 Data & Feature Management

The #1 silent production bug lives here: training-serving skew — nothing crashes, predictions just quietly get worse.

FEATURE STORE — ONE COMPUTATION, TWO CONSUMERS



Training-serving skew — training and serving compute the "same" feature via two different code paths that quietly drift apart. Nothing errors; predictions just degrade. Fix: one feature-store code path, used by both training and serving.

Two bugs that leak the future into training

- **Point-in-time join, skipped**

Joining a feature's *current* value instead of its value *at prediction time* — leaks future information into training.

- **Label leakage**

A feature accidentally encodes the label (e.g. `claim_approval_timestamp`) — looks perfect in validation, fails completely in production.

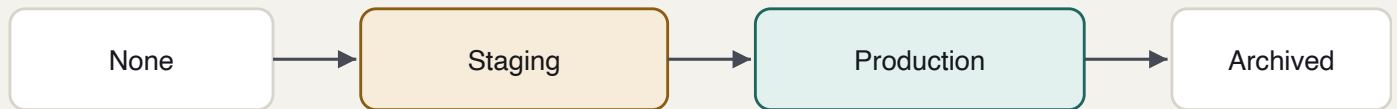
Data versioning: `dvc add` → small pointer file goes to Git, actual data goes to S3/GCS. `git checkout <commit> && dvc pull` reproduces the exact dataset. At scale: Delta Lake / Iceberg with native time travel.

04 Experiment Tracking & Registry

Dozens of training runs happen per model. Without tracking, "which one is best?" becomes unanswerable.

	SET BY	WHEN	DEVOPS ANALOGY
Parameters	Training process, automatically	During training	Compiled output itself
Hyperparameters	A human (or search)	Before training starts	Build flags (-o2, --target=prod)

MODEL REGISTRY STAGING WORKFLOW



```
# MLflow – track a run
with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.01)
    mlflow.log_metric("accuracy", 0.95)
    mlflow.sklearn.log_model(model, "my_model")

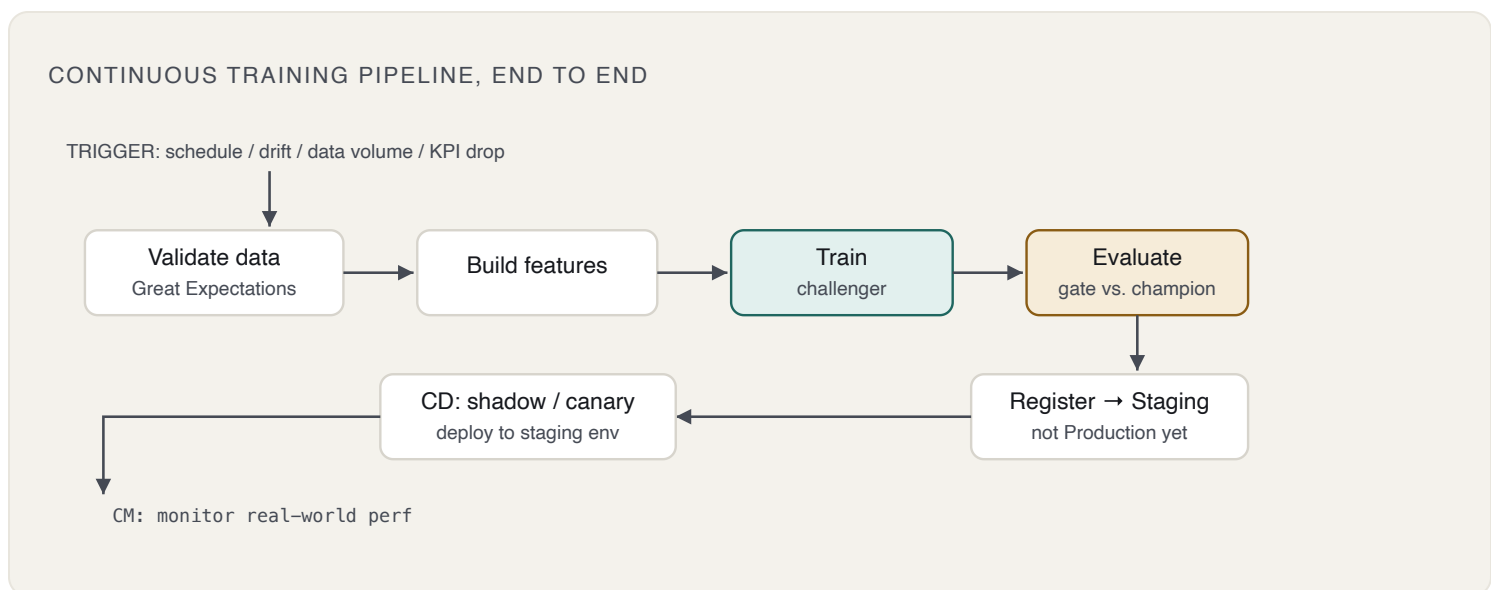
# promote a registered version
client.transition_model_version_stage(
    name="churn-predictor", version=3, stage="Production"
)
```

Model card: the README that ships with a registered model — intended use, training data, known limitations, fairness/bias results. Required wherever governance/compliance applies (finance, healthcare, hiring).

05 Pipelines — CI / CT / CD / CM

CT (Continuous Training) is the one pipeline type with no equivalent in traditional software — because a model can go stale with zero code changes.

PIPELINE	NORMAL SOFTWARE	ML
CI	Lint, unit test, build on every commit	+ validate data schemas every commit
CT	– doesn't exist –	Auto-retrain on trigger: new data, drift, schedule
CD	Deploy artifact if tests pass	Deploy new model if evaluation gates pass
CM	Uptime / latency / error-rate	+ drift & accuracy-decay monitoring



```
# evaluation_gate.py – runs after training, before promotion
if challenger.accuracy < champion.accuracy - 0.01:
    fail("regressed >1% vs champion")
if challenger.fairness_score < MIN_FAIRNESS_THRESHOLD:
    fail("failed fairness check")
register_model(new_model, stage="Staging") # never straight to Production
```

06 Deployment & Serving

A model can return 200 OK in 20ms and still be wrong. Rollout strategy exists to catch that before users do.

- **Batch**

Scheduled scoring over a large dataset. High throughput, latency irrelevant.

cron / ETL job

- **Online**

Synchronous HTTP/gRPC, usually <100ms.

tight-SLA REST API

- **Async**

Submit job, poll or webhook later.

SQS + worker

SAFE ROLLOUT FUNNEL — EACH STAGE COSTS MORE EXPOSURE, PROVES MORE CONFIDENCE



exposure increases →
confidence proven increases →

Rollback = point the serving layer at the previous registry version. If serving config is in Git (GitOps), rollback is literally `git revert` on the manifest.

07 Monitoring & Drift

Infra monitoring answers "is it up?" Model monitoring answers "are the predictions still good?" — neither substitutes for the other.

DATA DRIFT VS. CONCEPT DRIFT

Data drift

the questions changed



input distribution shifts
relationship input → output
stays the same

Concept drift

the answers changed



same-ish inputs, but the
input → output rule itself
has changed (e.g. new fraud tactics)

TEST	USE FOR	SIGNAL
PSI	Any feature, single summary number	<0.1 none · $0.1-0.25$ moderate · >0.25 investigate
KS test	Continuous numeric features	Are two samples from the same distribution?
Chi-squared	Categorical features	Category frequency shift
Jensen–Shannon	General distribution comparison	Symmetric, bounded divergence

Delayed-label problem: ground truth (e.g. "did the loan default?") may take months. Until then, watch proxies — confidence distribution shifts, output entropy jumps, input drift, and proxy business KPIs (CTR, conversion).

08 Platform, GPU & Cost

Your Kubernetes and cost-optimization instincts transfer directly — the one new variable is the GPU.

```
# GPU node pool: tainted so only GPU-requesting pods land here
spec:
  taints:
    - key: "nvidia.com/gpu"
      value: "true"
      effect: "NoSchedule"
# training job: tolerates the taint, requests the resource
resources:
  limits:
    nvidia.com/gpu: 1
```

LEVER	NORMAL DEVOPS	ML / GPU
Right-sizing	Don't over-provision CPU/memory	Don't request an A100 when a T4 is enough
Spot / preemptible	Batch jobs, CI runners	Training jobs, <i>with checkpointing</i>
Autoscale to zero	Idle dev environments	Idle GPU node pools
Off-peak scheduling	Nightly batch ETL	Large training runs in cheap pricing windows
Multi-tenancy	Bin-packing pods per node	NVIDIA MIG — split one GPU into isolated slices

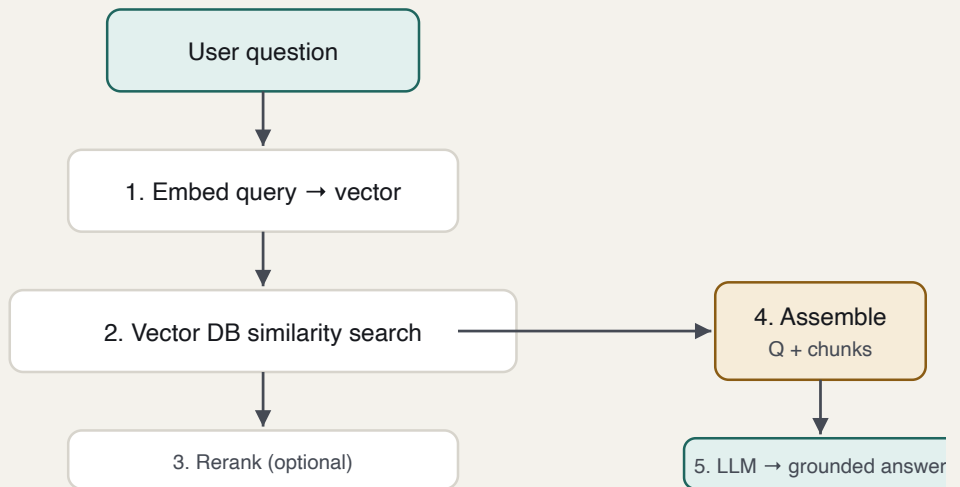
Checkpointing is what makes Spot instances safe for training: save model state every N steps to S3; an interrupted job resumes from the last checkpoint instead of restarting. Serving stays on stable capacity — a dropped inference pod means dropped user requests.

Distributed training, vocabulary only: data parallelism — full model copy per GPU, data shared · model/tensor parallelism — model itself sharded across GPUs · FSDP / DeepSpeed ZeRO — shard params, gradients, optimizer state.

09 LLMOps & GenAI

Everything above still applies. The shift: you usually aren't training the model — the operational focus moves to prompts, retrieval, and cost-per-token.

RAG — RETRIEVAL-AUGMENTED GENERATION



- **Faithfulness**

Does the answer only use facts present in retrieved context? (no hallucination)

- **Answer relevancy**

Does it actually address the question asked?

- **Context precision**

Of retrieved chunks, how many were actually relevant?

- **Context recall**

Did retrieval fetch *all* the relevant chunks?

Serving efficiently — vLLM

PagedAttention manages the per-request KV cache in small, non-contiguous pages — borrowing OS virtual-memory paging — eliminating fragmentation so far more concurrent requests fit per GPU. **Continuous batching** slots new requests into a running batch the instant a GPU slot frees up, instead of waiting for a fixed batch window.

Cost control

Model routing sends simple queries to a small/cheap model, complex ones to the large/expensive model. **Semantic caching** caches by *meaning* (embedding similarity), not exact text — serves a cached answer instead of calling the LLM again.

Condensed from the 17_MLOps reference set (00–09 + interview.md) · for personal study